Figure 13.1 Event logs include a timestamp, contextual information, and a message about what happened.

NOTE Spring Boot provides extensive options for configuring logging to files. Since that's not useful for cloud native applications, I won't cover it in this book. If you're interested in the subject, see the official documentation to learn more about log files (http://spring.io/projects/spring-boot).

ADDING LOGS TO SPRING BOOT APPLICATIONS

Besides configuring loggers for the frameworks and libraries used in your project, you should define event logs in your code whenever applicable. How much logging is enough? It depends on the context. In general, I reckon that it's better to have too much logging than too little. I've seen many deployments that just contain changes to add more logging, while it's pretty rare to see the opposite.

Thanks to the SLF4J façade, the syntax for defining new event logs in Java is the same no matter which logging library you use: a `Logger` instance created from a `LoggerFactory`. Let's see how it works by adding new log messages to the web controller in Catalog Service.

In the Catalog Service project (catalog-service), go to the `BookController` class, define a `Logger` instance from SLF4J, and add messages to be printed out whenever a client calls the application's REST API.

Listing 13.2 Defining log events using SL4FJ

```
package com.polarbookshop.catalogservice.web;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
...
```